Linux: tiến trình, quyền, service và điều tra lỗi

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Hệ điều hành và terminal

Hệ điều hành quản lý phần cứng và cung cấp môi trường cho chương trình chạy. Khi bạn mở một ứng dụng, hệ điều hành tạo tiến trình (process), cấp bộ nhớ và thời gian CPU, cho phép truy cập file và mạng theo quyền. Một server cũng là máy tính có hệ điều hành; khác biệt quan trọng là server thường chạy dịch vụ liên tục, nhận request từ máy khác và cần được theo dõi.

Đường dẫn cho biết vị trí file. Trên Linux, **/home/tenban/app/config.txt** dùng dấu **/** và bắt đầu từ gốc **/**. Trên Windows, **C:\Users\...** bắt đầu từ ổ đĩa. Thư mục hiện tại ảnh hưởng đến cách chương trình tìm file tương đối. Khi lệnh báo "không tìm thấy file", hãy kiểm tra thư mục hiện tại và đường dẫn trước khi sửa mã.

Trong Linux, quyền đọc (r), ghi (w) và thực thi (x) được áp cho chủ sở hữu, nhóm và người khác. Một script có thể tồn tại nhưng không được chạy vì thiếu quyền x, hoặc dịch vụ không đọc được cấu hình vì chủ sở hữu sai. Không dùng quyền root làm cách chữa mặc định: cần xác định tiến trình chạy dưới user nào và cấp quyền hẹp đủ dùng.

Service là tiến trình được quản lý để chạy nền. Trên nhiều bản phân phối Linux, **systemd** quản lý vòng đời service: khởi động, dừng, tự chạy lại và ghi log. Khi service lỗi, chuỗi suy luận là: trạng thái -> log -> cấu hình -> quyền -> phụ thuộc -> tài nguyên. Chỉ restart liên tục sẽ che dấu nguyên nhân.

Lab đọc-only trên Linux/WSL

Điều kiện: có terminal Linux hoặc WSL. Chạy từng lệnh và tự dự đoán kết quả trước khi xem: **pwd**, **ls -la**, **whoami**, **id**, **ps aux**, **df -h**, **free -h**. **pwd** cho vị trí hiện tại; **ls -la** cho file ẩn và quyền; **whoami/id** cho danh tính; **ps** cho tiến trình; hai lệnh cuối cho dung lượng đĩa và bộ nhớ. Một số môi trường WSL có thể hiển thị thông tin bộ nhớ khác máy Windows chính.

Ghi ít nhất ba phát hiện: user hiện tại là ai, thư mục hiện tại ở đâu, phân vùng nào chứa thư mục đó. Không cần gửi toàn bộ danh sách tiến trình hoặc thông tin nhạy cảm. Nếu máy chỉ có PowerShell, dùng **Get-Location**, **Get-ChildItem -Force**, **whoami**, **Get-Process**, **Get-PSDrive** để học cùng ý tưởng.

Tình huống điều tra

Dịch vụ không ghi được file log. Giả thuyết có thể là thư mục không tồn tại, quyền sai, đĩa đầy hoặc cấu hình log trỏ tới đường dẫn khác. Hãy viết thứ tự kiểm tra ít tác động nhất. Đáp án gợi ý: xác định user của service và đường dẫn log thực tế; xem quyền thư mục; xem dung lượng; đọc thông báo lỗi; chỉ sau đó thay đổi quyền hoặc cấu hình có kiểm soát.

Đọc một máy Linux theo lớp

Khi ứng dụng lỗi, bắt đầu từ dấu hiệu: người dùng không kết nối, kết nối được nhưng 5xx, hay request chậm. Kiểm tra process/service đang chạy (**ps**, **systemctl status** trên hệ dùng systemd), cổng lắng nghe (**ss -ltnp**), disk (**df -h**, **df -i**), memory (**free -h**), CPU/load (**uptime**, **top**) và log (**journalctl -u <service>**). Mỗi lệnh trả lời một câu hỏi khác. **df -h** còn dung lượng nhưng **df -i** hết inode vẫn có thể không tạo file mới. **systemctl active** chỉ cho biết service manager, không chứng minh API nghiệp vụ tốt.

Quyền file gồm owner, group và permission; process chạy dưới một user cụ thể. Nếu Notes API chạy user 10001 nhưng **/data** chỉ root được ghi, GET health có thể thành công còn POST ghi DB thất bại. Hỏi "process là user nào?", "đường ghi là gì?", "mount read-only không?" trước khi **chmod 777**. Quyền rộng có thể che lỗi cấu hình và tạo rủi ro dữ liệu. Với production, thay đổi owner/permission cần hiểu service khác có dùng cùng thư mục không.

Lab: ba lỗi ứng dụng cùng một triệu chứng

Chạy Notes API local trên máy lab. Trường hợp A: port 8000 đã có process khác; app không bind được. Dùng **ss -ltnp** hoặc công cụ OS tương đương để tìm process, không giết bừa nếu không phải lab. Trường hợp B: cấu hình **NOTES_DB** trỏ tới đường không writable; quan sát GET health và POST note, log Python. Trường hợp C: filesystem lab gần đầy (mô phỏng trong volume riêng, không làm đầy ổ hệ thống); quan sát lỗi ghi và inode/capacity. Với mỗi trường hợp ghi giả thuyết, bằng chứng, sửa và phép xác nhận. Sau lab trả lại port, đường DB và dung lượng như ban đầu.

Bash/PowerShell và tự động hóa an toàn

Một script vận hành cần nhận input rõ, kiểm tra lỗi, ghi log có ngữ cảnh và có thể chạy lại. Với Bash, phân biệt exit code của command và output của command; một lệnh thành công không đồng nghĩa cả chuỗi thao tác thành công nếu script nuốt lỗi. Tránh đặt secret trực tiếp trong command line vì có thể xuất hiện trong history/process list. Với PowerShell, đọc cmdlet **-WhatIf** khi hỗ trợ, kiểm tra **$ErrorActionPreference**/exception và dùng đường dẫn tuyệt đối cho thao tác file nhạy cảm. Chỉ chạy trên danh sách host/tài nguyên đã xác định.

Ví dụ script backup Notes API: kiểm tra file nguồn tồn tại, tạo tên backup chứa timestamp, copy tới thư mục lab, tính hash, restore thử vào file **khác**, rồi đọc dữ liệu. Nếu chỉ copy và in "success" nhưng không restore, chưa chứng minh khả năng phục hồi. Khi viết automation, luôn hỏi điều gì xảy ra nếu chạy lần hai, giữa chừng chết hoặc đầu vào thiếu.

Mô hình sâu: một request trên Linux đi qua những gì?

Một máy Linux không phải tập hợp lệnh để học thuộc. Khi client gọi HTTP tới Notes API, kernel nhận gói mạng, tìm socket đang lắng nghe, đánh thức tiến trình, tiến trình đọc cấu hình và file/DB, rồi ghi response. Nếu service không chạy, điều tra tiến trình và unit. Nếu process chạy nhưng không nghe cổng, xem bind address và socket. Nếu nghe cổng nhưng chỉ từ localhost, xem firewall/network. Nếu response 500, xem log ứng dụng và dependency. Sơ đồ này giúp bạn không dùng **chmod 777** hoặc restart máy mỗi khi gặp lỗi.

Tiến trình có PID, UID/GID, working directory, biến môi trường, file descriptor và giới hạn tài nguyên. **ps**, **/proc/<pid>**, **ss**, **lsof**, **journalctl** quan sát những mặt khác nhau của cùng tiến trình. Một process có thể vẫn **active** trong systemd nhưng không trả request đúng; systemd chỉ biết trạng thái tiến trình và điều kiện unit, không biết nghiệp vụ. Ngược lại, nếu service khởi động rồi thoát với exit code 1, trước tiên đọc **systemctl status** và journal theo thời điểm, sau đó mới thử chạy tay với cùng user/env để tái hiện.

Quyền file qua một ví dụ

Giả sử API chạy dưới user **notes**, cần đọc **/etc/notes/config.env** và ghi **/var/lib/notes/notes.db**. Đường tới file phải cho user đó quyền traverse (**x**) trên từng thư mục; file config cần quyền đọc; thư mục dữ liệu cần quyền ghi để tạo/đổi file và đôi khi file journal của SQLite. Quyền **644** trên DB nhưng thư mục không cho traverse vẫn lỗi. Root chạy được không chứng minh service user chạy được. Dùng **namei -l /var/lib/notes/notes.db**, **stat**, **id notes** và **sudo -u notes ...** trên VM lab để xác định lớp quyền. Tránh **chmod -R 777**: nó xóa ranh giới bảo vệ mà không tìm nguyên nhân.

Bộ nhớ, CPU và đĩa cần đọc đúng

CPU cao có thể là ứng dụng xử lý, vòng lặp lỗi, nén log hoặc DB query. CPU thấp mà latency cao gợi I/O wait, khóa DB, chờ mạng hoặc queue. **free -h** hiển thị cache/buffer; không coi mọi memory “used” là hết RAM. Disk có hai giới hạn khác nhau: byte (**df -h**) và inode (**df -i**). Một triệu file log nhỏ có thể hết inode dù còn GB trống. Khi DB không ghi được, kiểm tra filesystem mount read-only, dung lượng, inode và quyền trước khi kết luận SQL sai. **du** đo thư mục ở userland; file đã xóa nhưng process còn mở có thể vẫn chiếm chỗ đến khi file descriptor đóng.

Lab VM/WSL riêng: biến triệu chứng thành bằng chứng

Điều kiện: dùng Linux VM hoặc WSL riêng, không dùng máy production; có Python 3 và quyền đọc log của chính bạn. Không cần root cho phần đầu. Mở hai terminal. Ở terminal A tạo một thư mục tạm và chạy web server chỉ trên loopback:

```
mkdir -p ~/devops-linux-lab
cd ~/devops-linux-lab
python3 -m http.server 8765 --bind 127.0.0.1
```

Ở terminal B chạy **curl -i http://127.0.0.1:8765/**, **ss -ltnp** và **ps -ef | grep http.server**. **Kết quả dự kiến:** HTTP 200; socket **127.0.0.1:8765** LISTEN; process Python hiện trong danh sách. Không ghi PID cố định vào bài vì PID thay đổi. Hỏi: máy khác có gọi được cổng này không? Bind **127.0.0.1** chỉ nhận từ loopback của chính môi trường mạng đó; WSL có thêm lớp ánh xạ nên thử nghiệm từ Windows có thể khác Linux VM thuần.

Lỗi 1 - cổng bận: trong terminal C thử chạy server thứ hai cùng địa chỉ/cổng. Dự kiến lỗi bind “address already in use”. Đọc **ss** để biết ai sở hữu cổng; không kill một PID chỉ vì số cổng trùng trên máy dùng chung. Trong lab, Ctrl+C terminal A rồi chạy lại C. Giải thích: chỉ một listener phù hợp có thể bind tuple địa chỉ/cổng theo điều kiện thông thường.

Lỗi 2 - process dừng: Ctrl+C server, gọi lại curl. Dự kiến kết nối bị từ chối nhanh nếu không có listener; timeout sẽ gợi đường mạng/firewall hoặc packet bị bỏ. Phân biệt lỗi TCP này với HTTP 404: 404 nghĩa là đã tới một HTTP server nhưng tài nguyên không có.

Lỗi 3 - quyền đọc: tạo file **secret.txt** trong thư mục lab, đặt quyền **600**, rồi thử đọc bằng một user lab khác nếu có VM phù hợp. Nếu không có user thứ hai, chỉ đọc **stat** và dự đoán; đừng thay quyền hệ thống. Giải thích owner/group/other, quyền **x** trên thư mục, và vì sao chạy service bằng root để tránh lỗi là giải pháp kém.

Dọn: Ctrl+C tiến trình thử nghiệm; xóa đúng thư mục **~/devops-linux-lab** sau khi xác nhận nó là thư mục bạn vừa tạo. Không dùng wildcard xóa bên ngoài lab.

Systemd: đọc unit như hợp đồng khởi động

Một unit service thường chứa **ExecStart**, **User**, **WorkingDirectory**, **Environment** hoặc **EnvironmentFile**, **Restart**, dependencies và phần install. Nếu chạy tay tốt nhưng systemd lỗi, so **cùng binary**, **cùng working directory**, **cùng user**, **cùng env**, quyền file và thời điểm dependency. **After=network.target** chỉ định thứ tự; nó không luôn chứng minh network/DB đã sẵn sàng. **Restart=always** có thể tạo vòng lặp restart che lỗi cấu hình; xem restart count và log lần chạy trước.

Trên VM lab có systemd, đọc một service hệ thống vô hại bằng **systemctl cat <ten-service>** và **systemctl show <ten-service> -p User -p ExecStart -p Restart**. Không chỉnh unit hệ thống trong bài học. Tự phác một unit cho Notes API rồi trả lời: nếu file env thiếu thì log nào sẽ xuất hiện, service có quyền đọc DB không, và khi API trả 500 nhưng process sống thì systemd có phát hiện không?

Checklist điều tra có lời giải

Triệu chứng: HTTP timeout. Xem DNS/IP, TCP connect, listener **ss**, firewall, process; timeout chưa đủ để kết luận app lỗi. **Triệu chứng: Connection refused.** Trên IP đó không có listener phù hợp hoặc kết nối bị từ chối; kiểm tra bind address/cổng. **Triệu chứng: HTTP 500.** TCP/HTTP đã tới server; xem app log, dependency, quyền và tài nguyên. **Triệu chứng: process OOMKilled.** Xem kernel/cgroup signal, memory usage/limit và thay đổi tải; restart chỉ phục hồi tạm thời. **Triệu chứng: No space left on device.** Kiểm tra byte, inode, mount và file descriptor; không xóa log chưa rõ chủ sở hữu.

Bạn đạt bài này khi tự chỉ ra bằng chứng cho process, socket, quyền và filesystem; giải thích được vì sao cùng lệnh chạy tay và chạy qua service có thể khác nhau. Tài liệu gốc: systemd service, Linux proc filesystem.

Chuyên đề thực chiến: đọc log và tài nguyên theo thời gian

Một log line chỉ có ý nghĩa khi đặt trong timeline. Ví dụ 09:58 deploy bản B; 10:01 API 500 tăng; 10:02 systemd restart service ba lần; 10:03 disk **/var** còn 0 byte. Có thể B ghi log quá nhiều làm đầy **/var**, nhưng cũng có thể một job khác gây đầy trước rồi B chỉ là trùng thời điểm. Thu thập **journalctl** theo khoảng thời gian, **df -h**, **df -i**, kích thước thư mục log và thời điểm file tăng; so với mốc deploy. Đừng xóa log đang dùng trước khi ghi bằng chứng hoặc biết chính sách retention. Nếu filesystem báo read-only, xem **mount** và kernel log; **chmod** không chữa mount read-only.

Tiến trình chạy nhưng không phục vụ

Giả sử **systemctl status notes-api** báo **active (running)** nhưng **curl localhost:8000** bị từ chối. Kiểm tra unit **ExecStart**, process PID, **ss -ltnp** cho cổng 8000; có thể process đang nghe 8001 do biến môi trường đổi, hoặc bind Unix socket. Nếu **ss** có listener 127.0.0.1:8000 và localhost gọi được nhưng proxy từ container khác không tới, bind address/network namespace là nghi vấn. Nếu **ss** có 0.0.0.0:8000 mà curl trả 500, chuyển sang app log/DB, không tiếp tục sửa firewall. Đây là một decision tree: mỗi phép thử loại bỏ một lớp.

File descriptor và limit

Mỗi TCP socket/file mở chiếm file descriptor. Service có thể trả "too many open files" khi rò socket/log hoặc limit thấp. **ulimit -n** trong shell của bạn không nhất thiết là limit của service chạy qua systemd; xem unit/**/proc/<pid>/limits**. Đếm FD hiện tại của process trong lab và quan sát xu hướng, rồi tìm nguyên nhân rò trước khi tăng limit. Tăng limit có thể giúp tạm thời nhưng làm lỗi tăng chậm hơn và khó thấy. Một service với connection pool không đóng còn có thể làm DB cạn connection đồng thời.

Bài tập so sánh bốn lỗi giống nhau ở trình duyệt

A: DNS sai. Trình duyệt báo không tìm thấy host. Xác nhận **nslookup** và resolver path; **systemctl restart notes-api** không liên quan. **B: process chết.** IP đúng, TCP bị từ chối ở cổng; đọc service status và journal. **C: app bind loopback.** Trên host curl localhost được, từ proxy ở host khác không được; đọc **ss** và network path. **D: DB file không ghi được.** HTTP tới app, app trả 500 khi POST; đọc exception, owner/mode và filesystem. Hãy viết một phép thử chứng minh và một phép thử bác bỏ cho mỗi giả thuyết. Nếu chỉ viết "check logs", bài chưa đạt vì chưa nói log nào và kỳ vọng gì.

Shell script vận hành an toàn

Một script tự động dọn log cần xác định thư mục, owner, điều kiện xóa, dry-run và báo kết quả. Đường dẫn rỗng hoặc biến sai có thể làm tác động rộng; quote biến, kiểm tra thư mục tồn tại/thuộc phạm vi cho phép, tránh wildcard không kiểm soát. Trong Bash, kiểm tra exit status của lệnh và dừng/ghi lỗi đúng chỗ; trong PowerShell dùng **-LiteralPath** khi đường dẫn có ký tự đặc biệt. Không đặt mật khẩu trong đối số command line vì có thể lộ qua process listing/log. Script backup phải xác nhận file output tồn tại, checksum hợp lệ và restore thử được; "exit 0" không tự chứng minh dữ liệu dùng được.

Bài tập thiết kế: viết pseudocode cho script kiểm tra **/var/lib/notes** còn trên 20% dung lượng trống trước deploy, ghi tình trạng, từ chối deploy khi dưới ngưỡng, không tự xóa dữ liệu. Sau deploy gọi health endpoint và rollback nếu fail theo runbook. Nêu rủi ro TOCTOU: dung lượng có thể đổi giữa kiểm tra và deploy, nên vẫn cần metric/alert và xử lý lỗi ghi. Đây là sự khác nhau giữa automation có kiểm chứng và chỉ nối vài lệnh với **&&**.

Phỏng vấn sâu có đáp án

Vì sao free -h cho "used" cao chưa đủ kết luận hết RAM? Linux dùng RAM cho cache/buffer có thể thu hồi; xem available, swap/reclaim, PSI và tiến trình/cgroup. **Vì sao df -h còn chỗ mà app báo ENOSPC?** Có thể hết inode, quota, mount khác, hoặc filesystem riêng của container. **Vì sao root chạy được nhưng service user lỗi?** UID/GID, thư mục traverse, env, working directory và SELinux/AppArmor/policy có thể khác. **Vì sao restart chữa lỗi tạm thời nhưng không phải root cause?** Nó giải phóng state/connection/FD hoặc reset process, nhưng lỗi rò/giới hạn sẽ quay lại. Câu trả lời tốt nêu phép đo để kiểm tra mỗi khả năng.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://www.kernel.org/doc/
- https://www.freedesktop.org/software/systemd/man/latest/